

Relatório do Trabalho Prático

Processamento de Texto com Haskell

Paradigmas de Linguagens
Walace de Almeida Rodrigues

(Sem1, 2026)

Autora: Luisa Caetano Araujo

1 Introdução

Este relatório descreve o desenvolvimento de um programa em Haskell para análise de frequência de palavras em textos. O programa lê um arquivo de texto, divide-o em parágrafos e, para cada parágrafo, exibe uma lista de pares (palavra, frequência) ordenados por frequência decrescente.

O trabalho foi desenvolvido com foco na aplicação de conceitos de programação funcional, como funções puras, imutabilidade e composição de funções.

2 Decisões de Implementação

2.1 Estrutura Geral

O código foi organizado em seções bem definidas, separando claramente a lógica pura das operações de entrada e saída (IO). Essa separação facilita a manutenção, os testes e a compreensão do código.

2.2 Tratamento de Acentuação

Para normalizar palavras com e sem acentos, foi implementada a função `removeAccent`, que mapeia caracteres acentuados para seus equivalentes ASCII. Por exemplo, “ação”, “Ação” e “ACAO” são tratadas como a mesma palavra (“acao”).

```

1 removeAccent :: Char -> Char
2 removeAccent c = case c of
3   ' ' -> 'a'; ' ' -> 'a'; ' ' -> 'a'; ' ' -> 'a'
4   ' ' -> 'e'; ' ' -> 'e'
5   ' ' -> 'c'
6   ...
7   _ -> toLower c

```

2.3 Stopwords

As stopwords (palavras comuns a serem ignoradas) foram armazenadas em um `Set` de `String`, estrutura que oferece busca eficiente em $O(\log n)$. A lista inclui artigos, preposições, conjunções e pronomes em português.

2.4 Divisão em Parágrafos

Um parágrafo é definido como um bloco de texto separado por uma ou mais linhas em branco. A função `splitParagraphs` implementa essa lógica dividindo o texto em linhas e agrupando linhas não vazias consecutivas.

3 Bibliotecas e Estruturas de Dados

Foram utilizadas apenas bibliotecas da base padrão do Haskell:

- **Data.Char**: funções `toLower` e `isAlpha` para manipulação de caracteres.
- **Data.List**: funções `sortBy` e `groupBy` para ordenação e agrupamento de listas.
- **Data.Set**: estrutura de conjunto para armazenamento eficiente das stopwords.
- **System.Environment**: função `getArgs` para leitura de argumentos da linha de comando.

As principais estruturas de dados utilizadas foram:

- **WordFreq**: tipo sinônimo para tupla `(String, Int)`, representando palavra e frequência.
- **Paragraph**: tipo sinônimo para `[String]`, representando uma lista de linhas.
- **Set String**: conjunto de stopwords para filtragem eficiente.

4 Descrição das Funções

4.1 Funções de Normalização

- **removeAccent**: converte caracteres acentuados para ASCII e aplica lowercase.
- **isLetter**: verifica se um caractere é uma letra (incluindo acentuadas).
- **normalizeWord**: aplica **removeAccent** a cada caractere de uma palavra.
- **processWord**: remove pontuação e normaliza a palavra.

4.2 Funções de Processamento

- **splitParagraphs**: divide o texto em parágrafos com base em linhas em branco.
- **splitOn**: função auxiliar genérica para dividir listas com base em predicado.
- **extractWords**: extrai todas as palavras de um parágrafo.
- **filterStopwords**: remove stopwords e palavras vazias da lista.
- **countFrequencies**: conta a frequência de cada palavra usando **groupBy**.
- **sortByFrequency**: ordena por frequência decrescente (empates em ordem alfabética).
- **processParagraph**: pipeline completo que combina todas as etapas de processamento.

4.3 Funções de Formatação

- **formatWordFreq**: formata um par (palavra, frequência) para exibição.
- **formatParagraphOutput**: formata a saída completa de um parágrafo.
- **formatOutput**: combina a saída de todos os parágrafos.

4.4 Função Principal

A função **main** é a única que realiza operações de IO:

1. Lê o nome do arquivo dos argumentos da linha de comando.
2. Lê o conteúdo do arquivo.
3. Aplica o processamento (funções puras).
4. Exibe o resultado formatado.

5 Principais Aprendizados

Durante o desenvolvimento deste trabalho, os principais aprendizados foram:

- **Separação IO/Lógica:** a importância de isolar operações de entrada e saída das funções puras, facilitando testes e manutenção.
- **Composição de funções:** como combinar funções simples para construir pipelines de processamento complexos.
- **Pattern matching:** uso extensivo para tratar diferentes casos de forma clara e segura.
- **Estruturas imutáveis:** como trabalhar com dados imutáveis e transformações funcionais.
- **Uso de Set:** eficiência de conjuntos para operações de busca frequentes.

6 Dificuldades Encontradas

As principais dificuldades encontradas foram:

- **Tratamento de acentos:** mapear todos os caracteres acentuados do português para seus equivalentes ASCII exigiu atenção aos detalhes.
- **Divisão em parágrafos:** definir corretamente o que constitui uma linha em branco (espaços, tabs) e implementar a lógica de agrupamento.
- **Ordenação com critérios múltiplos:** ordenar por frequência decrescente e, em caso de empate, alfabeticamente.

7 Conclusão

O trabalho permitiu aplicar na prática os conceitos de programação funcional estudados na disciplina. O programa desenvolvido atende a todos os requisitos especificados, processando textos em português com suporte a acentuação e filtragem de stopwords.

A experiência reforçou a importância da organização do código em funções pequenas e bem definidas, característica central do paradigma funcional.